ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA
CESENA CAMPUS

Department of Computer Science and Engineering - DISI
Second-cycle Degree in Digital Transformation Management
Class: LM-91

Semantic Foundations for Conversational Analytics: Grounding LLM Agents in Material Planning at Amadori

Graduation thesis in
Laboratory on Digital Transformation Technologies

Supervisor
Prof. Matteo Golfarelli

Candidate
Francesca Folli

3rd Session
Academic Year 2025-2026

Abstract

Acknowledgements

Contents

Abstract **iii**

1 Introduction **1**

2 Corporate Profile and Strategic Overview of Amadori S.p.A. **3**
2.1 Industrial Profile 3
2.2 Portfolio Strategy 4
2.3 Operational Constraints 5
2.4 Demand Planning Rationale 6

3 The Forecasting Engine and Material Planning Project **7**
3.1 Overview of Statistical Models for Demand Forecasting 7
3.2 The "As-Is" Demand Model: ARIMAX and TSLM Approaches . . 8
3.3 The Material Planning Engine 9

4 Architectural and Organizational Evolution **13**
4.1 The Data Product Paradigm 13
4.2 Conceptual Alignment 14
4.3 The Semantic Layer 15
4.4 Enterprise Knowledge Graphs 17
4.5 Active Metadata Management 18
4.6 Organizational Transformation 19

5 The Technological Framework and Software Stack **21**
5.1 Backend Infrastructure 21
5.2 The Semantic Layer with Looker and LookML 23
5.2.1 Architectural best practices and performance optimization . 23
5.2.2 Centralized governance and the DRY principle in Security Design 24
5.2.3 Implementation: Material Planning fact and master views . 25
5.2.4 Model definition: relational joins and access perimeter . . . 30

5.2.5 Development lifecycle and version control 33
5.2.6 Semantic modeling for self-service exploration and AI enablement 34
5.3 Metadata Architecture Roadmap 37

6 Conversational Analytics and Agentic Workflows with Gemini in Looker 41
6.1 Introduction 41
6.2 Looker Conversational Modalities 42
6.3 System Instructions and Domain Contextualization 44
6.4 Golden Queries 46
6.5 Iterative Tuning of the Agent 48
6.6 Visualization Limits 50
6.7 Impact on the Data Office 52

List of Figures

2.1 The Vertically Integrated Supply Chain Architecture of Amadori S.p.A. 4

3.1 Material planning demand decision tree: chronological data source selection, product archetype routing (PFL vs. PN/PFN), platform allocation, and sequential BOM explosion. 10

5.1 Overview of the technological stack for the Material Planning domain. Solid lines show the current prototype setup; dashed elements represent the planned phases (direct Looker–Oracle connection, migration to production BigQuery) and the OpenMetadata governance extension. 22

5.2 The *Fabbisogno lordo materiali* Explore in Looker: the field picker exposes only the curated business namespaces defined in the LookML model, while technical identifiers remain hidden. 33

5.3 Self-service exploration in Looker: gross ingredient requirements for a single platform, broken down by product line and forecast quarter. 35

5.4 The exploration of Figure 5.3 saved as a reusable Look (*Materials by line and quarter*), consumed by business users in read-only mode; platform, material type, and unit of measure remain adjustable filters. 36

6.1 *Chat with Dashboard* in Looker: the Gemini side panel opened on a Material Planning dashboard, with suggested starter questions. . . . 43

6.2 *Chat with Explore* in Looker: the conversation is started from the *Fabbisogno lordo materiali* Explore through the *Start a conversation* button (highlighted). 43

6.3 Bar chart with a linear scale produced by the agent: the largest item hides all the others. 51

6.4 Chart requested with a logarithmic scale: the answer states that it was generated, but the chart is empty. 51

Listings

5.1 LookML definition for Material Planning fact view (partial excerpt) 25
5.2 LookML definition for standard Platform master view (partial excerpt) . . . 27
5.3 LookML master Articles view (representative excerpt) 28
5.4 LookML Model and Explore configuration for Material Planning (simplified excerpt) . . . 30
5.5 Proposed LookML configuration for single unit-of-measure selection (not yet implemented) . . . 32
6.1 Excerpt of the agent's system instructions (default behaviors and filters), as configured in Looker . . . 45

Chapter 6

Conversational Analytics and Agentic Workflows with Gemini in Looker

6.1 Introduction and Business Motivation

The semantic layer described in Chapter 5 gives the Material Planning domain a single, certified definition of its metrics, and the Explore, Looks, and dashboards built on it already allow Data & AI Champions to answer many questions without the central data team. Self-service exploration, however, still requires users to know the model: which fields to select, which filters to apply, and how to read the result. For planners, procurement managers, and plant supervisors who need a quick answer to a specific question (for example, how much of a given packaging material a plant will need next month), this remains a barrier.

This chapter describes how the barrier was lowered by introducing a conversational agent built with the Conversational Analytics feature of **Gemini in Looker**. The agent is not used as a generic Text-to-SQL generator working on raw tables, an approach prone to wrong joins, incorrect aggregations, and hallucinated field names. Instead, it is bound to the LookML model of Section 5.2: it can only use the certified dimensions, measures, and joins of the `material_planning` Explore, and every query it produces is compiled by Looker and subject to the same access rules as any other query.

Binding the agent to the semantic layer ensures that metrics are computed according to the certified definitions, but it does not ensure that the agent always chooses the right fields and filters. Most of the work described in this chapter addresses this second problem. Section 6.2 introduces the conversational modalities available in Looker and the one adopted. Section 6.3 describes the system instruc-

tions that encode domain knowledge and operational defaults, and Section 6.4 the golden queries that complement them with reference examples. Section 6.5 presents the iterative tuning based on the agent's reasoning traces and the resulting migration of business logic upstream, Section 6.6 discusses the limits observed in the charts produced by the agent, and Section 6.7 discusses the impact on the role of the Data Office.

6.2 Conversational Modalities in Looker

Looker offers three ways to query data in natural language [11]:

1. **Chat with Dashboard:** an assistant opened from a specific dashboard. It answers questions within the context of that dashboard and its tiles, summarizing results and comparing tiles, and can also query the Explores on which the dashboard is built.

2. **Chat with Explore:** an assistant started from a single Explore through the *Start a conversation* button. It translates a question into dimensions, measures, and filters of that Explore, and is mainly intended for users who already work in self-service.

3. **Conversational Analytics data agents:** named, persistent agents created by a developer and shared with users [12]. Each agent can be connected to up to five Explores and has its own instructions describing the business context. When *Advanced Analytics* is enabled, the agent can also translate a question into Python code and execute it, for analyses and charts that go beyond a single query.

Figures 6.1 and 6.2 show the first two modalities on the Material Planning assets. In the first, the side panel is opened on a dashboard built from the Look of Figure 5.4 and proposes starter questions, such as the main takeaways or the filters currently applied. In the second, the conversation is started directly from the `material_planning` Explore, labeled *Fabbisogno lordo materiali* in the Looker interface.

The third modality was adopted for the Material Planning use case, for three reasons. First, only a data agent can be given persistent instructions: the domain taxonomy, defaults, and rules described in Section 6.3 would otherwise have to be repeated by each user in every conversation. The same holds for the golden queries of Section 6.4, which are supported only by data agents. Second, the agent is a single governed asset: it is defined once, tested, and shared, so that all users query the data through the same validated configuration. Third, the agent combines Advanced Analytics with the persistent instructions, so that requests such

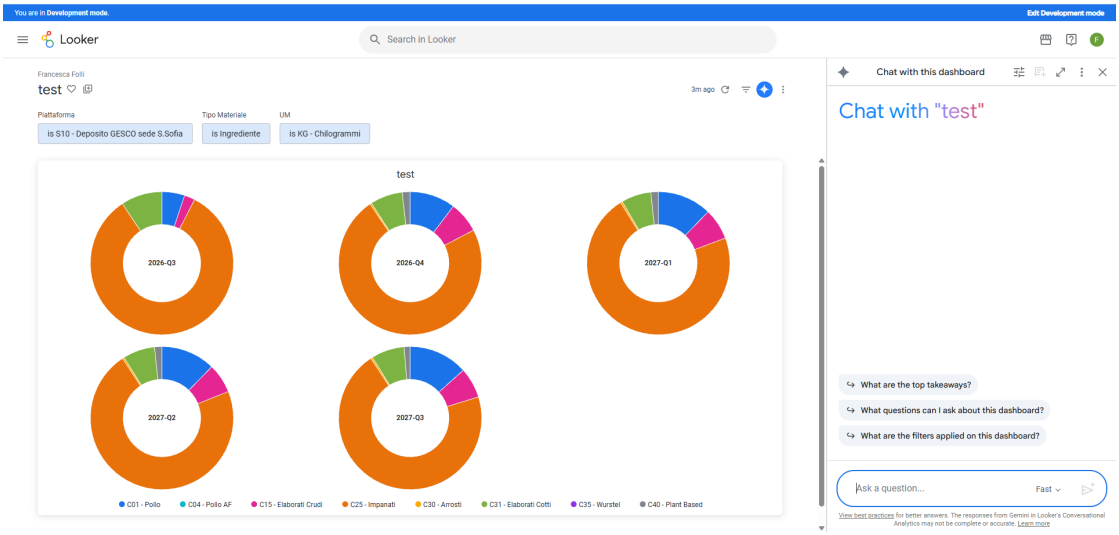


Figure 6.1: *Chat with Dashboard* in Looker: the Gemini side panel opened on a Material Planning dashboard, with suggested starter questions.

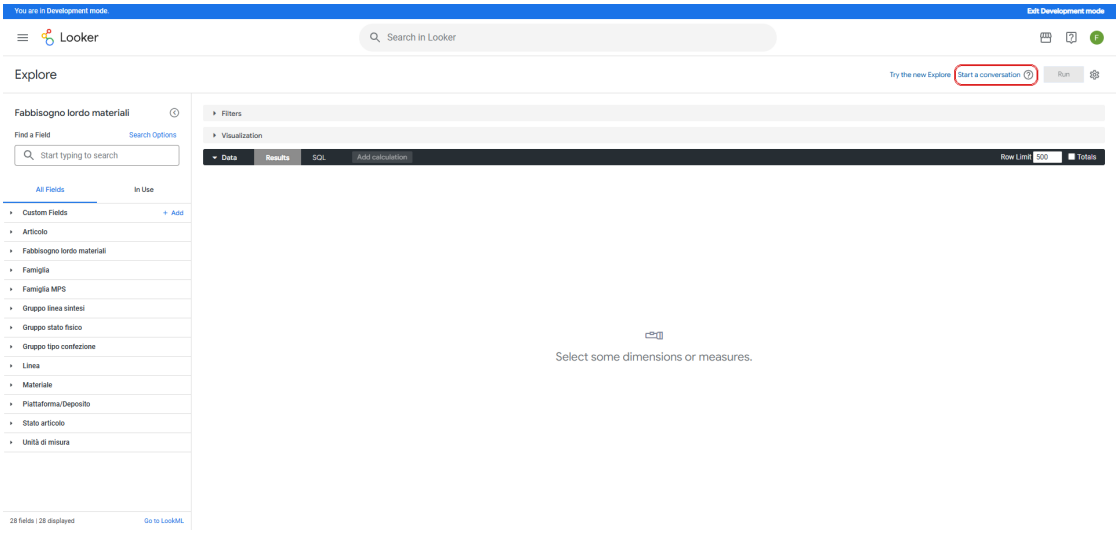


Figure 6.2: *Chat with Explore* in Looker: the conversation is started from the *Fabbisogno lordo materiali* Explore through the *Start a conversation* button (highlighted).

as peak detection or statistical analysis, which a single Looker query cannot answer, are also handled according to the domain rules; its use for charts is discussed in Section 6.6. The agent is connected to the `material_planning` Explore and, depending on the question, answers with text, a native Looker chart, or a chart produced in Python.

6.3 System Instructions: Codifying Enterprise Domain Knowledge

The LookML model tells the agent which fields exist and how they are computed, but not how the Material Planning team talks about its data, which values to assume when a question is incomplete, or how results should be presented. This knowledge is provided through the agent's **system instructions**, which are organized in four blocks: role and response policy, business terminology, default behaviors and filters, and visualization guidelines. The first three are described in this section; the visualization guidelines are discussed in Section 6.6.

Role and response policy The agent is defined as an expert data assistant for Amadori's Material Planning, with a professional, analytical, and concise tone. Two rules govern its answers: they must be direct and based on data, without unnecessary preambles, and every answer must end with two or three suggested follow-up questions or next analytical steps. The second rule turns a single answer into a guided exploration: after a question on total requirements, for example, the agent may suggest breaking the result down by material type or by platform.

Business terminology The instructions define the terms that planners use and link them, where possible, to fields of the model:

- **Gross requirement (*fabbisogno lordo*):** the total quantity of material needed, derived from finished product demand, corresponding to the measure `fabbisogno_materiale`.
- **PFL (*prodotto finito in linea*):** a labeled, market-ready product whose BOM contains the complete set of ingredients, primary packaging, and labels. Mapped through synonyms to the dimension `prodotto.prodotto`.
- **PN (*prodotto neutro*):** an unlabeled physical item whose associated BOM includes only ingredients and primary packaging. Mapped through synonyms to the dimension `prodotto.articolo_neutro_cod`.

- **PFN (*prodotto finito da neutro*):** the commercial variations derived from a Neutral Product, with a BOM containing exclusively the specific finished label(s). Mapped through synonyms to the dimension `prodotto.prodotto`.

These definitions, already introduced in Chapter 3, allow the agent to interpret questions such as "how many neutral products require this packaging?" without the user having to know which field encodes the concept.

Default behaviors and filters Most planning questions are incomplete: they do not specify a measure, a platform, or a time horizon. Rather than asking the user each time, the instructions define explicit defaults (Listing 6.1):

- **Default measure:** words such as "fabbisogno", "quantità", or "domanda" are always mapped to `fabbisogno_materiale`; `casse_articolo` is used for finished product volumes.
- **Default platform perimeter:** when a question refers to platforms generically, or does not mention any, the query is restricted to the platforms relevant to material planning. The instructions present this filter as an absolute requirement.
- **Default time horizon:** operational questions without a horizon cover the period from today to the end of the following month; analytical or strategic questions cover the whole available forecast horizon.
- **Exact match on codes:** codes provided by the user are treated as exact matches. This rule was added during the tuning process and is discussed in Section 6.5.

Listing 6.1: Excerpt of the agent's system instructions (default behaviors and filters), as configured in Looker

```
# COMPORTAMENTI PREDEFINITI E LOGICA DEI FILTRI
Misura Predefinita:
  - Quando gli utenti menzionano fabbisogni, quantità o domanda ("fabbisogno",
    "quantità", "domanda"), calcola sempre utilizzando la misura
    `f_material_planning.fabbisogno_materiale`.
  - Misura secondaria per il volume dei prodotti finiti:
    `f_material_planning.casse_articolo`.

Filtro Piattaforma Predefinito (Obbligatorio):
  - Applica sempre un filtro fisso su `f_material_planning.piattaforma_cod`
    IN ("S01", "S02", "S04", "S05", "S10", "S50") ogni volta che sono coinvolti
    depositi/piattaforme o quando non viene specificata una piattaforma
```

```
    specifica. Consideralo un requisito assoluto.

Filtro Data Predefinito:
  - Query operative / standard: Se non è specificato un orizzonte temporale,
    filtra `f_material_planning.data_previsione_date` dalla data odierna alla
    fine del mese successivo.
  - Query di analisi / strategiche: Utilizza come intervallo di date tutto
    l'orizzonte di previsione disponibile.
```

Limitations and future improvements Some domain knowledge is currently handled only in the instructions, or not handled at all, because the model does not yet support it. These cases are candidates for the same upstream migration described in Section 6.5:

- **Platform perimeter as a model constraint:** the default perimeter is enforced only by the agent's instructions, so it does not apply to Explore users and depends on the agent following them. Declaring it in the Explore as an `always_filter` with the same default values would make it part of the model and apply it to every consumer, while still allowing users to change it explicitly.
- **Identification of labels:** labels cannot currently be distinguished reliably from other packaging materials. Recognizing them from text patterns in the material description proved too weak, since short strings such as "Et" also occur in unrelated descriptions. A robust solution requires a dedicated attribute in the item master data, brought into the backend as was done for the material type (Section 6.5).

6.4 Golden Queries: Teaching the Agent by Example

The system instructions describe the domain through general rules. Looker data agents offer a complementary mechanism: **golden queries**, called *verified queries* in the Looker interface [12]. A golden query pairs a question in natural language with the Explore query that answers it correctly. The developer writes the question and links the URL of an Explore in which fields, filters, and visualization have already been set. These pairs act as reference examples: when a user asks a similar question, the agent can rely on a verified mapping between business terms and the fields and filters of the model, instead of inferring it.

The documentation sets some constraints. Golden queries are available only for data agents, not for Chat with Explore or Chat with Dashboard; they must

refer to an Explore connected to the agent; they do not support pivots or custom fields. No more than 30 to 50 pairs per agent are recommended, covering different types of questions and a variety of filters and filter values.

Four golden queries were defined for the Material Planning agent (Table 6.1). The questions, originally in Italian, are translated here. Each one covers terms or rules that the agent could not derive from the model alone.

ID	Question	Filters	What it teaches
GQ-1	*"What is the gross requirement split between ingredients and packaging for the next 3 months?"*	Forecast month from the current month for 3 months.	"Ingredients" and "packaging" as values of the material type; results grouped by month and unit of measure.
GQ-2	*"How many trays do I need in Santa Sofia for the next 5 days?"*	Platform `S10`; material contains "Vas"; forecast date from today for 5 days.	Place name resolved to a platform code; "trays" resolved from the material description.
GQ-3	*"Considering only Elaborati, how many packaging materials do I need until the end of the year?"*	Material type *Imballo*; product line *Elaborati*; forecast date until the end of the current year.	Product line and material type combined; results by material and unit of measure.
GQ-4	*"What is the expected consumption of I02165 in Cesena?"*	Platform `S01`; material starts with `I02165`.	"Consumption" as gross requirement; code matched at the start of the material field; daily trend.

Table 6.1: Golden queries defined for the Material Planning agent.

Mapping business terms The first three queries show the agent how planners name the categories of the model. In GQ-1 and GQ-3, "ingredients" and "packaging" are mapped to the values of `tipo_materiale`, the dimension introduced in Section 6.5, and in GQ-3 "Elaborati" is mapped to the product line. In GQ-2, the name of the plant is resolved to its platform code, and "trays" (*vassoi*) to the materials whose description contains "Vas", which is how planners themselves search for trays in the Explore.

Codes and units of measure GQ-4 complements the exact-match rule of Section 6.5. Since the material code is stored together with its description in the

dimension `materiale.materiale`, the query filters on the values that *start with* `I02165`, rather than on those that contain it. The word "consumption" is mapped to the gross requirement measure. Since the question asks for an analysis of a single material and does not specify a horizon, the query covers the whole available forecast horizon, by day. GQ-1 and GQ-3 also show how to handle units of measure: they group the results by unit, so that quantities expressed in different units are never added together.

Visualization Each golden query also fixes how the answer is presented: horizontal bars by month and type in GQ-1, bars by material in GQ-2 and GQ-3, and a line over time in GQ-4. For these questions, the presentation is therefore defined in advance, which partly offsets the limits discussed in Section 6.6.

Limitations Golden queries are examples, not rules: they guide the agent on similar questions but do not guarantee its behavior on different ones, so they complement the instructions rather than replace them. They also require maintenance, since every change to the model or to the instructions has to be reflected in the Explore queries they point to.

6.5 Iterative Tuning and Upstream Migration of Business Logic

The agent was not configured in a single step. Its behavior was refined through repeated test questions, using the **Show Thinking** option of Looker's Conversational Analytics. For each answer, Show Thinking displays the agent's reasoning: how it interpreted the question, which fields and filters it chose, and which query or Python code it ran. Reading these traces made it possible to see where the agent was guessing and to correct the cause, either in the instructions or in the data model. Two cases are described below: in the first, the correction was made in the data model; in the second, in the instructions.

From agent heuristics to the backend: the material type In the first tests, a question such as *"Split ingredients from packaging"* could not be answered with a field of the model, because the Explore did not yet expose a material type. The agent did not stop there: its Show Thinking trace revealed that it retrieved the material codes and used Python to infer a classification from their structure:

```
"Codes starting with '0' seem to represent ingredients
(Budello, Olio), while codes starting with 'I' represent
```

```
packaging (Film, Vassoio)...  Retrieving raw codes to run
python_agent categorization..."
```

The inference was plausible, but it had three problems. It was not an official rule, only a pattern guessed from the data. It was recomputed in every conversation, so the result depended on how the agent interpreted the codes each time. And it required retrieving and processing many rows, which made the answer slower than a normal query.

The official classification, however, already exists in JD Edwards, in the item master data. It was therefore added to the exported master table `L_ART_JDE` as the column `TIPO_MAT`, and exposed in LookML as a simple mapping to that column, the `tipo_materiale` dimension shown in Listing 5.3. After the change, the same question is answered with a single query that groups by `tipo_materiale`, without Python.

This case shows a general rule, consistent with the *pushing transformations upstream* practice of Section 5.2: the agent should not compute business rules on its own. When its reasoning reveals a missing concept, the concept should be defined in the data model and, when possible, taken from the source system. The same approach is proposed for the identification of labels, discussed among the limitations at the end of Section 6.3.

Exact matches and the superset trap A second problem appeared with questions containing codes, such as *"How many ingredients are needed for item 00700?"*. Instead of running the query, the agent stopped and asked whether the user meant `00700` or one of the codes that contain it, such as `00700F` or `100200700`. This behavior, which the agent's own reasoning calls the **superset trap**, is reasonable for vague text, but not for a code that the user has written exactly.

The following rule was added to the default behaviors of the instructions:

> *"Se l'utente fornisce un codice (articolo, materiale, piattaforma o unità di misura), questo deve essere considerato una corrispondenza esatta (EXACT_MATCH). Se il codice inserito è presente nel database, non applicare la logica 'Superset Trap' e non richiedere chiarimenti su codici simili o che lo contengano."*[1]

With this rule, when a code exists in the data the agent filters on it directly, and asks for clarification only when the code is not found.

[1] "If the user provides a code (item, material, platform, or unit of measure), it must be treated as an exact match (EXACT_MATCH). If the code is present in the database, do not apply the 'Superset Trap' logic and do not ask for clarification about similar codes or codes that contain it."

6.6 Visualization Limits

The tests showed a clear gap between the agent's reasoning and its charts. The agent follows the instructions when it interprets a question, chooses fields and filters, and computes results. Its charts, however, are reliable only when they are simple and use a linear scale.

What the documentation states Conversational Analytics draws charts with Vega-Lite [11, 24]. Line, area, bar, scatter, and pie charts are supported, and each question must be answerable by a single visualization. The documentation does not explain how to set scales or axes. Advanced Analytics can draw charts in Python, with libraries such as Altair and matplotlib, but its answers may be more variable [10].

What the tests showed Material requirements are hard to chart: a few materials often account for most of the total, so on a linear scale they flatten all the others. The instructions ask the agent to show such data in a table, and to use a logarithmic scale only when the user asks for it. In the tests, a request for a chart of all ingredients in Cesena produced a single bar chart with a linear scale, where the largest item hides the others (Figure 6.3). An explicit request for a logarithmic scale produced an empty chart, although the answer stated that the chart had been generated correctly (Figure 6.4). The same question also gave a table in one session and a chart in another. Conversely, when explicitly asked to produce one chart for each unit of measure, the agent returned several charts in the same answer. Table 6.2 compares these observations with the documentation.

Capability	**Documentation**	**Observed**
Simple chart, linear scale	Supported.	Works, but unreadable with unbalanced data.
Logarithmic scale	Not described.	Empty chart.
Several charts in one answer	One visualization per question.	Produced when explicitly requested (one chart per unit of measure).
Consistency across sessions	Answers may vary (Advanced Analytics).	Same question gave a table in one session and a chart in another.

Table 6.2: Visualization capabilities: documentation compared with the tests.

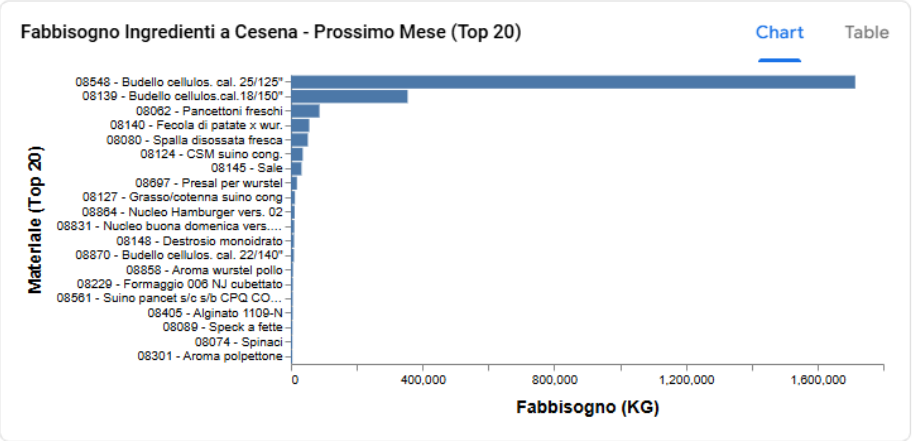


Figure 6.3: Bar chart with a linear scale produced by the agent: the largest item hides all the others.

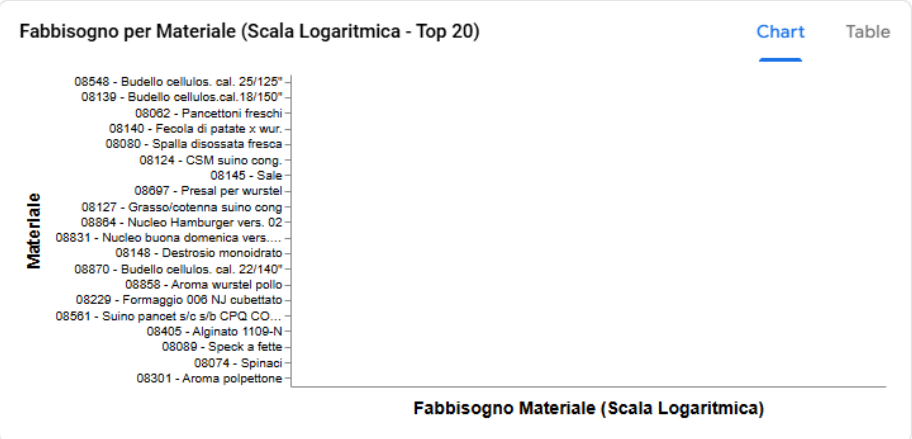


Figure 6.4: Chart requested with a logarithmic scale: the answer states that it was generated, but the chart is empty.

Implications These limits cannot be fixed through the instructions, which shape the agent's reasoning and text but not the rendering of its charts. The agent should therefore be used mainly for what it does well: understanding business questions, applying the right definitions and filters, and explaining the results. Charts should be checked before they are trusted, and charts that must be read regularly are better built as Looks and dashboards, where their settings are fixed.

6.7 Impact on the Role of the Data Office

Section 4.6 described the shift of the Data Office from a service team that builds reports on request to a governance team that defines and certifies shared semantic assets. The conversational agent makes this shift concrete, and adds new tasks to it.

For planners, supported by the Data & AI Champions, the agent removes the need to know the model: questions can be asked in business vocabulary, and the answers are computed on the certified definitions of the semantic layer. The constraints that are not yet part of the model, such as the platform perimeter and the exact match on codes, are applied through the instructions, and therefore depend on the agent following them; this is why the answers, and especially the charts, still need to be read critically.

For the Data Office, the work moves from answering individual requests to maintaining the context in which the agent operates:

- **Instructions and golden queries:** the terminology, defaults, and reference examples of Sections 6.3 and 6.4 have to be kept aligned with the model and with the way planners actually ask questions.
- **Reading the reasoning traces:** Show Thinking reveals where the agent is guessing, as in the cases of Section 6.5, and is the main source for deciding what to correct.
- **Moving business logic upstream:** when a trace reveals a missing concept, the Data Office defines it in the data model and, where possible, takes it from the source system, as was done for the material type and is planned for labels.

In this way, the agent does not replace the semantic layer but depends on it: the more business concepts are defined in the model, the fewer rules the agent has to infer, and the more reliable its answers become.

Bibliography

[1] Christine L. Borgman. *Big Data, Little Data, No Data: Scholarship in the Networked World*. MIT Press, Cambridge, MA, 2015.

[2] Geoffrey C. Bowker and Susan Leigh Star. *Sorting Things Out: Classification and Its Consequences*. MIT Press, Cambridge, MA, 1999.

[3] George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco, CA, 1970.

[4] DAMA International. *The DAMA Guide to the Data Management Body of Knowledge (DAMA-DMBOK2 Revised)*. DAMA International, revised edition, 2024.

[5] Thomas H. Davenport and Jeanne G. Harris. *Competing on Analytics: The New Science of Winning*. Harvard Business Review Press, Boston, MA, updated edition, 2017.

[6] Zhamak Dehghani. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media, Sebastopol, CA, 2022.

[7] Darren Edge, Ha Trinh, Aidan Newman, Joshua Christopher, Andrew Evans, Shenghao Wang, Kevin Larson, Robert Foster, Kyle Bennett, Scott Jones, et al. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[8] Google Cloud. Best practices for configuring conversational analytics and ai agents in looker. `https://cloud.google.com/looker/docs/conversational-analytics-looker-best-practices`, 2024. Accessed: 2026-10-15.

[9] Google Cloud. Lookml and explore best practices for high-performance queries. `https://cloud.google.com/looker/docs/best-practices`, 2024. Accessed: 2026-10-15.

[10] Google Cloud. Advanced analytics in conversational analytics. https://docs.cloud.google.com/looker/docs/conversational-analytics-looker-advanced-analytics, 2026. Accessed: 2026-11-05.

[11] Google Cloud. Conversational analytics in looker overview. https://docs.cloud.google.com/looker/docs/conversational-analytics-overview, 2026. Accessed: 2026-11-04.

[12] Google Cloud. Create and manage explore data agents in looker conversational analytics. https://cloud.google.com/looker/docs/conversational-analytics-looker-data-agents, 2026. Accessed: 2026-10-15.

[13] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

[14] Joseph Hilger, Lulit Tesfaye, and Zachary Wahl. *Bridging Knowledge, Data, and AI: Harnessing the Semantic Layer Framework to Drive Intelligence*. Professional and Applied Computing Series. Springer Nature, Cham, Switzerland, 1st edition, 2026.

[15] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, José Emilio Labra Gayo, Sabrina Kirrane, Sebastian Neumaier, Axel Polleres, Roberto Navigli, Axel-Cyrille Ngonga Ngomo, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

[16] Rob Kitchin. *The Data Revolution: Big Data, Open Data, Data Infrastructures and Their Consequences*. SAGE Publications, London, 2014.

[17] Sabina Leonelli. *Data-Centric Biology: A Philosophical Study*. University of Chicago Press, Chicago, IL, 11 2016.

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[19] Ikujiro Nonaka and Hirotaka Takeuchi. *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, New York, NY, 1995.

[20] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. Technical Report KSL-01-05, Stanford Knowledge Systems Laboratory, Stanford, CA, 2001.

[21] Dip Bharatbhai Patel. Comparative analysis of data visualization tools: Tableau, power bi & looker. *The Eastasouth Journal of Information System and Computer Science*, 1(1):55–59, 2023.

[22] Pranitha Potturi. Demystifying semantic layers in business intelligence platforms. *Sarcouncil Journal of Engineering and Computer Sciences*, 4(8):612–618, 2025.

[23] James Richardson, Kurt Schlegel, Rita Sallam, Austin Kronz, and Julian Sun. Magic quadrant for analytics and business intelligence platforms. Technical Report G00467317, Gartner, February 2021.

[24] UW Interactive Data Lab. Vega-lite: A grammar of interactive graphics. `https://vega.github.io/vega-lite/`, 2026. Accessed: 2026-11-05.